数值分析—非线性方程组上机报告

数80 应怀原

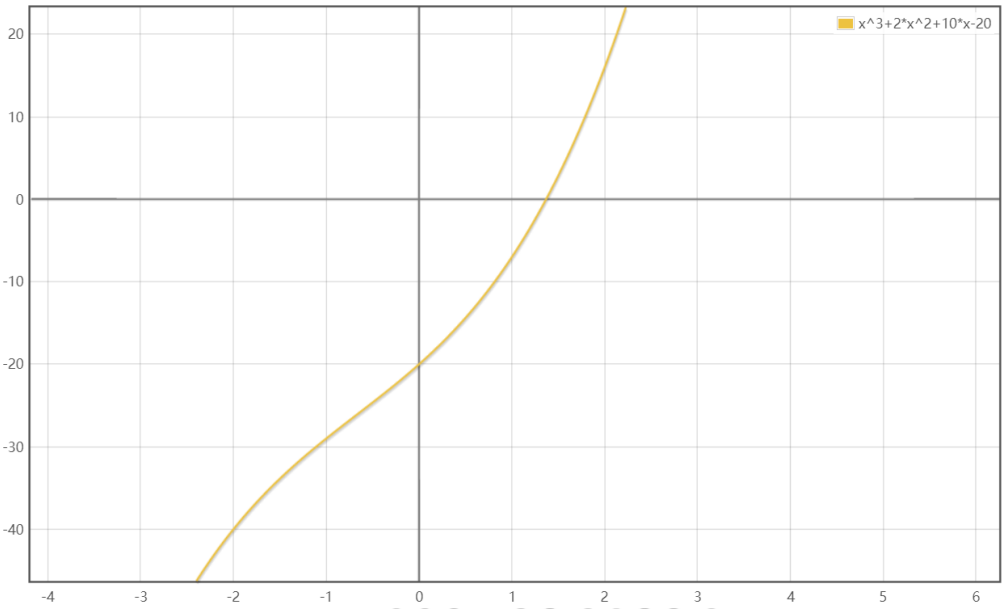
2020 年 10 月 24 日

1 实验目的

本次上机作业需要用迭代法解决非线性方程组的问题。用到的方法有：不动点迭代法（两个不同的迭代函数），Steffensen加速法和newton迭代法。所要求的方程为：

$$x^3 + 2x^2 + 10x - 20 = 0$$

直观起见，先画出函数图像，并用计算器进行计算：



得到$x^* \approx 1.368808107\cdots$是唯一一个解。事实上，记上式左边为$f(x)$，得$f'(x) = 3x^2 + 4x + 10$，判别式小于0，于是$f'(x) > 0$恒成立，这给出根的唯一性。

我们期望通过这次作业，比较几种方法直接的效果、收敛速度等，并期望求出至少7位有效数字的解。

2 实验方法详述

2.1 不动点迭代法

作业文件中提供了两个不动点迭代法，其一是将原方程改写成：

$$10x = 20 - x^3 - 2x^2$$

两边除以10得到函数$f_1(x) = \frac{20-x^3-2x^2}{10}$，于是迭代就是解方程$f_1(x) = x$，取$x_0 = 1$，$x_{k+1} = f_1(x_k)$。

但我们从理论上考虑，在x^*处，我们有$f_1'(x^*) = -\frac{3(x^*)^2+4x^*}{10} \approx 1.1096 > 1$，因此这不是一个压缩映射，迭代法有可能不收敛。

第二种方法，将原方程改写成：

$$x^3 = 20 - 10x - 2x^2$$

两边开三次方得到函数$f_2(x) = \sqrt[3]{20 - 10x - 2x^2}$，同第一种方法，取$x_0 = 1$，$x_{k+1} = f_2(x_k)$。

在x^*处，我们有

$$f_2'(x^*) = \frac{-10 - 4x^*}{3\sqrt[3]{20 - 10x^* - 2(x^*)^2}} \approx -3.76854$$

因此也有可能不收敛。

在实际编程过程中，我们首先定义func-1和func-2分别是上述的迭代函数f_1和f_2，定义函数shoulian-1和shoulian-2，输入为初始值$x_0 = 1$，控制用误差$e = 10^{-10}$。

每次迭代后，判断$x_k^3 + 2x_k^2 + 10x_k - 20$的绝对值是否小于$e$，如果是则停止迭代并返回所求出的$x$和迭代次数。由于上述分析，算法可能不收敛，因此还需要取停止时间$t = 10000$，即10000次迭代还未收敛到上述值则直接返回。

2.2 Steffensen加速法迭代

我们沿用上一部分中函数f_1和f_2的记号，来叙述Steffensen加速法。具体算法如下：

在每一次迭代中，除了x_k之外，还需引入y, z满足：

$$y_k = f_i(x_k), z_k = f_i(y_k)$$

随后一次迭代就是

$$x_{k+1} = x_k - \frac{(y_k - x_k)^2}{z_k + x_k - 2y_k}$$

根据课上所讲的定理，只要$f_1'(x^*)$和$f_2'(x^*)$不为1，则对应的迭代方法至少是二阶收敛的。由第一部分中的求导分析，这个条件是满足的。因此理论上此方法是收敛的。

实际编程过程中，我们定义函数Steffensen-1和Steffensen-2，并考虑如前所述的迭代步骤。和上一部分一样，输入为初始值$x_0 = 1$，控制用误差$e = 10^{-10}$。每次迭代后，判断$x_k^3 + 2x_k^2 + 10x_k - 20$的绝对值是否小于$e$，如果是则停止迭代并返回所求出的$x$和迭代次数。虽然理论上收敛，但考虑到数值因素，我们还是设定最大迭代次数为$t = 10000$。

2.3 Newton迭代法

我们沿用第一大section的记号$f(x) = x^3 + 2x^2 + 10x - 20$，按照牛顿迭代法的要求，一次迭代为：

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

仍然是从理论上考虑，我们有

$$f''(x) = 6x + 4, f'(x) = 3x^2 + 4x + 10$$

首先$f(x)$在$[0,2]$上有零点，其次$f''(x)$在$[0,2]$上不变号，最后$|f'(0)| = \min\{|f'(0)|, |f'(2)|\}$满足

$$10 = \frac{|f(0)|}{2-0} = |f'(0)|$$

因此根据课上讲的定理，牛顿迭代法二阶收敛到根x^*。

同样地我们有迭代步骤、初始值、控制用误差和最大迭代次数。由于数值求导是不稳定的，我们手动计算了f的导数，是多项式，并在程序中定义了f-deri函数以便调用。

3 实验结果讨论

在本次结果所使用的图片中，x_i代表用作业第i小问的方法得到的结果，time_i则是用作业第i小问的方法的迭代次数。最大迭代次数为10000。

3.1 不动点迭代法

在我们的初始设定下，$x_0 = 1, e = 10^{-10}$，我们得到的结果为左图。由于差距实在很大，我重新取值$x_0 = 1.368808$，即初始值离真值非常近，得到的是右图。

```
x_1 0.548946478054766 time_1 10000
x_2 -3.162277660168379 time_2 10000
```

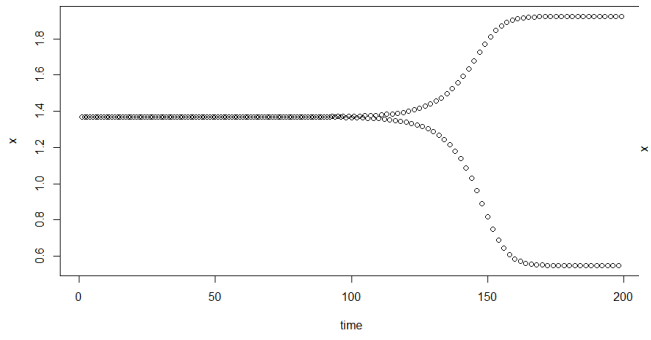
(a) x0 = 1

```
x_1 0.548946478054766 time_1 10000
x_2 -3.162277660168379 time_2 10000
```

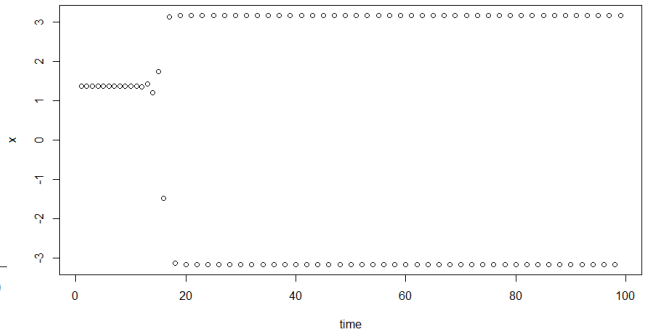
(b) x0 = 1.368808

图 1: Results

令人惊讶的是，不论初值距离真值有多近，得到的答案居然是一样的。为了更好的看清，我们对前100次迭代，两种不同迭代函数得到的值做散点图。



(a) method1



(b) method2

图 2: scatter plot

可以看出的是，两种方法都发散了，且存在对应方法i的数a_i, b_i满足$a_i \neq b_i$，$f_i(a_i) = b_i, f_i(b_i) = a_i, i = 1, 2$。两种方法最后得到了这样的平衡，从而不会收敛到真正的根。

3.2 Steffensen加速法迭代

对初始值$x_0 = 1$，同样的控制误差$e = 10^{-10}$，我们得到结果左图，如果进一步控制误差$e = 10^{-20}$，得到右图

```
x_3 1.3688081078213719 time_3 4
x_4 1.3688081078213725 time_4 8
```

(a) e= 1e-10

```
x_3 1.3688081078213727 time_3 5
x_4 1.3688081078213725 time_4 8
```

(b) e= 1e-20

图 3: Results

两种方法的效果都很好，收敛到了我们想要的具体的根。在误差允许较大的时候，第一种方法迭代了四次输出结果，第二种方法迭代了8次，在误差缩小的时候，第一种方法迭代了5次。我试验到$e = 10^{-200}$，两种方法还是得到这个结果，因此理论上不可再改进。为了进一步看到两种方法的收敛，我将两种方法迭代每一步的结果都输出：

```
3 1.3334921391138637 1
3 1.3684154391083934 2
3 1.3688080583091646 3
3 1.3688081078213719 4
3 1.3688081078213727 5
```

(a) method3

```
4 1.2 1
4 1.2737403995453735 2
4 1.3083000390078872 3
4 1.347275219739435 4
4 1.3667239138143357 5
4 1.3687892848827672 6
4 1.3688081062896424 7
4 1.3688081078213725 8
```

(b) method4

图 4: Results

从中可以看出，第一种迭代函数（即方法3）在第三次迭代就拥有了7位有效数字，第二种迭代函数（即方法4）在第六次迭代拥有5位有效数字，到第七次迭代才拥有7位有效数字，相对而言要慢一些。

这一发现和我们上一个小节中看到的有类似现象，第二个迭代函数，在采用简单不动点迭代的时候，发散的更快。这都和我们在第二大节中算出的导数的绝对值有关系。导数绝对值越大，发散的越快。

为了测试初值敏感性，我们取了不同的x_0，列表如下。

x_0	1	2	3	0	-1	-3
迭代次数	5	5	6	5	6	6

表 1: 第一种迭代函数

x_0	1	2	3	0	-1	-3
迭代次数	8	10	11	10	10	11

表 2: 第二种迭代函数

随着初值离真值越来越远，迭代的次数也越来越多，输出结果则是保持几乎不变。但第一种迭代函数仍然是优于第二种的。当x_0更大的时候，python会报错‘Result too large’，我认为这是因为不收敛导致的迭代序列以每次立方的速度发散了。

3.3 Newton迭代法

和上一节一样，我们将newton迭代法的每次结果都输出：

```
5 1.4117647058823528 1
5 1.3693364705882352 2
5 1.3688081886175318 3
5 1.3688081078213745 4
5 1.3688081078213727 5
```

(a) method5

图 5: Results

它也是在第三步拥有了7位有效数字。在$e = 10^{-10}$的条件下，newton迭代法迭代四次即输出，在$e = 10^{-20}$的条件下，newton迭代法迭代5次才会输出结果。这时候的结果和method3完全一致，与method4均相差一位。

我们同样考虑对初值的敏感性，根据之前的理论推导，至少在$[0, 2]$内应当是收敛的。

x_0	1	2	3	0	-1	-3
迭代次数	5	5	6	6	6	7

表 3: Newton迭代

首先在$[0, 2]$内确实是收敛的，在$[-3, 3]$以外（只考虑整数的话），和Steffensen加速法一样，会导致序列发散。同时，和Steffensen加速法相比，newton方法在偏离较远时不如第一种迭代函数收敛的快。

4 实验总结

通过这次实验我们可以看出，为了让数值求解非线性方程的方法切实可行，我们首先需要选取理论上收敛的方法，在这些方法中，我认为Steffensen加速法的判定最为简单，Newton法稍显复杂，而一般的不动点方法需要严格验证才能使用，不然便会出现问题。

同时，不同的迭代方法，迭代函数，其数值精度和收敛速度也有很大的差异，在更大规模的计算中，我们应当先估计收敛速度和精度，再使用方法。同时，一个好的初值可以很大程度上加速运算，因此直观地给出根的区间，以便确定初值大致范围，也是有必要的。